Parsing Is Not Executing: Decentralized Compliance for Agentic Query Plan Routing

Ranjan Sinha
rsinha@us.ibm.com
IBM
San Jose, California, USA

Abstract

An engine that parses a Substrait plan is not necessarily one that executes it correctly. We learned this the hard way while building the Substrait Compliance Framework, a decentralized system for testing query plan interoperability across heterogeneous engines. The framework ships multi-language SDKs (Java, Python, Rust), 1,635 function-level test cases spanning 15 categories, and serialized TPC-H and TPC-DS benchmark plans with CI/CD-ready reporting. Testing three production engines turned up results we did not expect: one achieves 22/22 syntactic TPC-H acceptance but falls short on full semantic validation, function-level pass rates of 94% overall mask dramatic per-category swings, and a prototype evidence-aware routing agent reduced semantic mismatches from 15 to 5 on a 50-item mixed workload (66.7% reduction) while exposing a string-processing category where no available engine meets the quality threshold.

CCS Concepts

• **Information systems** → **Query languages**; • **Software and its engineering** → *Interoperability*; • **Computing methodologies** → *Artificial intelligence.*

Keywords

Substrait, interoperability testing, decentralized compliance, AI agents, data systems, query plan portability, federated governance

ACM Reference Format:
Ranjan Sinha. 2026. Parsing Is Not Executing: Decentralized Compliance for Agentic Query Plan Routing. In *Proceedings of Supporting Our AI Overlords Workshop at CAIS 2026 (SAO 2026).* ACM, New York, NY, USA, 5 pages.

1 Introduction

Who writes your analytical queries? Increasingly, the answer is not a person. AI agents now select data sources, construct query plans, orchestrate multi-engine pipelines, and interpret results with minimal human oversight [2, 17]. This shift matters for infrastructure because agents operating at pipeline speed do not stop to inspect whether a `DECIMAL` rounded differently than expected, do not cross-reference results against domain intuition, and rarely validate outputs before passing them downstream. They trust the interchange format. When that trust is misplaced, errors compound silently.

Substrait [16] defines a portable intermediate representation for relational algebra, decoupling plan *producers* from *consumers*. The promise is clear: write a plan once, execute it faithfully anywhere. But SQL's decades of dialect fragmentation [4, 6] suggest caution. Even at the plan level, engines can parse a Substrait plan correctly while producing wrong results on aggregation edge cases, window function framing, or implicit casts [11]. Parsing is not executing.

This paper describes the Substrait Compliance Framework, a system we built to make these divergences visible. We report on testing three production Substrait consumers (anonymized as Engine A, B, and C) against 1,635 function-level tests and benchmark queries from TPC-H and TPC-DS. The findings surprised us in places. Engine B went from passing 2 of 22 TPC-H queries to all 22 after integrating the framework into its CI pipeline. Engine C parses every TPC-H plan without error yet does not achieve full semantic fidelity. And a naive routing agent that ignores compliance evidence produces 15 semantic mismatches on a 50-item workload; one that consults it reduces this to 5, while flagging a string-processing category where the ecosystem itself falls short.

2 The Interoperability Gap

Data systems were designed for careful human operators who understand engine quirks, write queries in specific dialects, and manually verify results [17]. An agent composing an analytical workflow cannot do any of this. It expresses a plan once and expects faithful execution across whichever Substrait consumer it targets. The agent will not inspect type coercion rules or null propagation semantics; it relies on the interchange format entirely.

Why is this reliance risky? Because syntactic agreement has never guaranteed semantic equivalence in query processing [10]. Casting behavior, function definitions, null handling, optimizer assumptions: all vary across engines even when the surface syntax is shared [6]. Substrait operates at the plan level rather than the query text level, which helps, but the deeper challenge carries over.

Agentic workloads make this worse. An agent in an automated pipeline does not squint at the output and notice that revenue looks off; it propagates errors silently. And agents increasingly compose plans that span multiple engines in a single workflow. Every engine boundary is another chance for semantics to diverge. When agents generate high-throughput, speculative query variants [17], those mismatches compound fast.

One might argue that centralized conformance testing (a single authority validating all engines) would solve the problem. In practice, it does not scale well. It creates bottlenecks, conflicts with deployment constraints (some engines require local infrastructure

SAO 2026, San Jose, California

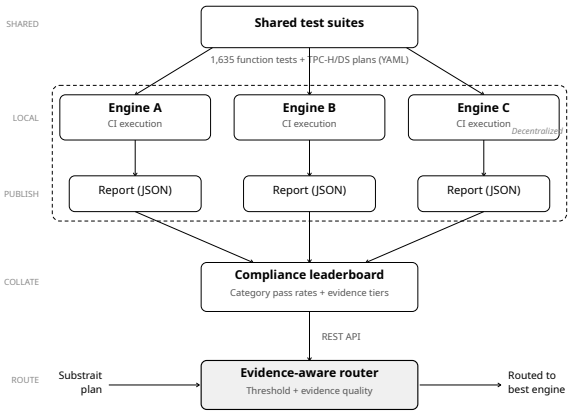

Figure 1: Compliance framework workflow. Shared test suites are executed locally by each engine within its own CI pipeline (dashed region). Structured reports feed a public leaderboard, which the evidence-aware router queries before dispatching Substrait plans to the best-matching engine.

or proprietary connectors), handles sensitive diagnostic artifacts poorly [3], and reduces conformance to a binary badge when agents actually need category-level granularity. Lakehouse architectures with pipeline-level atomicity [18] reinforce the point: testing belongs inside existing CI pipelines, not in an external service.

3 The Substrait Compliance Framework

We built a decentralized compliance framework with three layers: shared community artifacts (versioned test suites, metadata conventions, reporting schemas), engine-local execution, and standardized result publication. Figure 1 shows the end-to-end pipeline, from shared test suites through engine-local CI execution and structured report publication to the compliance leaderboard, which the evidence-aware router queries before dispatching each plan.

3.1 Multi-Language SDK

The framework ships SDKs in Java, Python, and Rust. Each provides an idiomatic implementation of a common contract: Java uses interfaces, Python uses abstract base classes, Rust uses traits. Engine developers pull the SDK into their own repository and implement the `ComplianceEngine` interface. The interface has four methods. Two are metadata: `getEngineInfo()` for engine name, version, and vendor, and `getCapabilities()` for declaring supported relations, functions, and types through an `EngineCapabilities` descriptor. The other two do the real work: `validatePlan()` checks structural validity of a Substrait plan, and `executePlan()` runs it and returns tabular results. All execution happens locally. No data leaves the engine team's environment.

Internally, the SDK uses a four-layer architecture with dependencies pointing inward (infrastructure, domain, application, presentation). Domain objects are immutable, which simplified concurrent test execution; we observed 3–4× speedups on multi-core runs without locking. The `YamlTestSuiteLoader` loads plan data lazily, and the `TestSuiteLoader` interface is pluggable for future suite formats.

The capability declaration system turned out to be more important than we initially expected. An engine that does not implement geospatial functions declares this via `EngineCapabilities`, and the runner skips those tests automatically. Results carry four states: `PASSED`, `FAILED`, `SKIPPED`, `ERROR`. Without this distinction, a leaderboard conflates absence with failure and penalizes engines that report honestly.

3.2 Test Suite

The framework includes two complementary suites.

Function-level tests. 1,635 test cases across 111 files in 15+ categories: arithmetic (475 tests, 45 functions including trigonometric and bitwise operations), string (319 tests, 28 functions), comparison (183), datetime (180), boolean (108), aggregate (97, covering `count_distinct`, `median`, `variance`), window (95, for ranking/framing/partition semantics), array (94), cast (88), set (45), geospatial (40), conditional (35), map (31), JSON (30), and struct (21). Each test specifies input data, a Substrait plan targeting a specific function with defined edge conditions (nulls, overflow, empty groups), and expected output with explicit floating-point tolerances.

Benchmark queries. 22 TPC-H queries [19] and 5 initial TPC-DS queries [13], serialized as Substrait plans in binary and JSON formats. TPC-H uses scale factor 0.01 (86,630 rows across 8 tables); TPC-DS uses scale factor 0.01 (288,000 rows, 10 tables, three sales channels). Queries range from simple to very complex. Critically, these are *semantic fidelity* tests, not performance benchmarks. We measure correctness, not throughput [5].

3.3 Developer Workflow

We optimized for low friction. An engine team picks the SDK matching their language, implements `ComplianceEngine`, loads the test suite, runs it through `ComplianceRunner`, and wires it into CI/CD with our GitHub Actions template (one file, five variables). Optionally, results go to the leaderboard. The whole process from `git clone` to first passing test takes under five minutes.

Results flow through a hierarchical aggregation pipeline into a `ComplianceReport` (per-category pass rates, exportable as JSON, Markdown, HTML, or CSV). The public leaderboard sits behind a REST API with JWT authentication, rate limiting, and webhook support for downstream notifications. It separates TPC-H results from function-level results and assigns status tiers (Excellent, Good, Fair) based on pass rates.

4 Empirical Results

We ran three production Substrait consumers against the full suite.[1] A fourth (Engine D) is under integration but not yet ready for inclusion.

4.1 TPC-H

Engine A and Engine B both achieve 22/22 (100%) on semantic validation. Engine B's trajectory is worth noting: it started at 2/22

[1]Engine identifiers are anonymized. All three are mature, open-source analytical engines with active Substrait integration.

before the compliance framework was integrated into its development workflow. That progression, from near-total failure to full compliance, suggests the framework does not just measure quality but can drive it. Engine C tells a different story. It parses all 22 plans via its Calcite-based [1] Substrait path and passes 5/5 in a focused semantic suite, but full semantic coverage lags behind Engine A and Engine B. An engine that accepts every plan is not the same as one that executes every plan correctly. For agents, syntactic acceptance without semantic verification is arguably worse than outright rejection, because it creates a false signal of correctness.

4.2 TPC-DS

Engine A and Engine C both reach 87/99 (87.9%) on TPC-DS syntactic coverage; all 87 pass structural validation against minimal data. Engine B is not currently the primary TPC-DS validation target. The 12 failures are interesting. They cluster around recursive CTEs, certain `ROLLUP`/`GROUPING SETS` patterns, and advanced temporal arithmetic. We suspect several of these reflect specification ambiguity rather than implementation bugs.

4.3 Function-Level Coverage

The aggregate pass rate across all engines is 546/581 (94.0%). That number is real but misleading if taken at face value. Categories with mature Substrait extension definitions (arithmetic, comparison, boolean) reach near-complete coverage. Categories involving string operations tell a different story: Engine B passes only 56.6% of 957 string tests, and Engine A hovers around 60%. This is not an outlier. Categories involving newer or less-specified extensions (geospatial, JSON, map) show similarly low pass rates and wide inter-engine divergence. An engine passing 100% of boolean tests and 57% of string tests looks very different depending on what your plan needs. An agent composing a plan that calls `concat` and `regexp_replace` should not assume the same level of fidelity as one calling `and` and `or`. The aggregate score hides exactly this distinction.

4.4 Complementary Strengths

No engine dominates every dimension. Engine A provides the strongest semantic TPC-H reference path and broad TPC-DS coverage. Engine B serves as the primary function-level executor with strong overall coverage. Engine C contributes multi-engine path validation and TPC-DS structural testing. This is a complementarity that rewards routing strategies aware of it and punishes those that assume one engine handles everything equally well.

5 Agents and Compliance

5.1 Three Roles

The most straightforward use of the leaderboard is for routing. An agent queries category-level pass rates through the REST API, identifies which engines have known weaknesses in specific semantic areas, and avoids routing plans through gaps in coverage. Tagliabue et al. [17] describe this as the "data systems for agents" direction; our compliance API fits that vision.

But agents can also sit on the other side: producing conformance evidence rather than consuming it. Embedded in an engine's CI pipeline, an agent runs the suite on every commit, detects regressions, and submits structured reports. The GitHub Actions template automates this. Quality gates fail builds when pass rates drop below configurable thresholds, catching semantic regressions before release. In lakehouse architectures with data branching [18], these runs can be isolated on a branch so test artifacts never touch production data.

A third role is more speculative. An agent monitoring query telemetry could notice that production plans heavily exercise `INTERVAL` arithmetic, which happens to be underrepresented in the test suite, and flag the gap. We have not built this yet, but the plugin architecture was designed with it in mind; extension points for suites, loaders, reporters, and validators accept community contributions without core SDK changes.

5.2 Routing Evaluation

We benchmarked compliance across TPC-H (22 queries, 86K rows), TPC-DS (99 queries), and four function categories (1,480+ test cases), then fed the measured pass rates into a prototype evidence-aware router. The evaluation workload has 50 items: 20 TPC-H queries, 10 TPC-DS queries, 10 arithmetic tests, 5 string tests, and 5 boolean tests. The naive router sends everything to Engine B. The evidence-aware router picks the engine with the highest pass rate above an 85% threshold, preferring real integration evidence over extrapolated estimates.

Table 1: Compliance evidence driving routing decisions. Engine B's TPC-DS score is extrapolated; all other scores are from real integration testing.

Category	Eng. A	Eng. B	Selected
TPC-H	100% R	100% R	A (20)
TPC-DS	**87.9%** R	75% E	A (10)
Arith.	90% R	**95%** R	B (10)
String	60% R	56.6% R	A (5)†
Bool.	100% R	100% R	B (5)
Total distribution			A:35 B:15

R=REAL_INT. E=EXTRAP. †Below threshold

The results (Tables 1 and 2) tell a more nuanced story than a clean sweep would. The naive policy produced 15 mismatches. The evidence-aware router reduced this to 5, a 66.7% improvement. All 10 avoided mismatches were TPC-DS queries: Engine B's TPC-DS score of 75% was extrapolated from its TPC-H performance (the actual TPC-DS test suite has not yet been run against it), and the router correctly redirected these to Engine A (87.9%, real integration). For arithmetic, Engine B won legitimately (95% vs. 90%, both real integration). For boolean, both engines scored 100%.

The 5 remaining mismatches are string tests, and they expose a genuine limitation. Engine A passes 60% of string tests (on 200+ cases); Engine B passes 56.6% (on 957 cases). Both fall below the 85% threshold. The router selected Engine A as the lesser of two problems, but it cannot fix a category where no available engine is adequate. This is an honest result: compliance-aware routing helps where the evidence discriminates between engines, but it cannot conjure quality that does not exist. The string category is a gap the

Table 2: Mismatch avoidance by workload category.

Category	Items	Naive	Aware	Avoided
TPC-H	20	0	0	0
TPC-DS	10	10	0	10
Arith.	10	0	0	0
String	5	5	5	0
Boolean	5	0	0	0
Total	**50**	**15**	**5**	**10**

Substrait ecosystem needs to close at the implementation level, not the routing level.

The evidence-aware policy selected Engine A for 70% of queries and Engine B for 30%, compared to 100% Engine B under naive routing.

6 Related Work

The lakehouse architecture [20] created the multi-engine topology that makes plan-level interoperability a practical concern. Work on lakehouse storage formats [7] surfaced analogous challenges between Delta Lake, Iceberg, and Hudi; we face the same class of problem at the query plan layer. Velox [12] took the shared-runtime approach: one execution library embedded across systems. Our model assumes independently developed engines, matching the Substrait ecosystem and the broader trend toward specialized engines [15]. Shankar et al. [14] showed that continuous validation outperforms one-time certification in ML pipelines; the same principle applies to plan semantics. TPC-DS [9, 13] provides richer query structures than TPC-H and motivates our ongoing suite expansion.

Database conformance testing predates our work by decades. NIST maintained SQL suites, and JDBC/ODBC specifications include their own tests. What we do differently is test at the *plan level* rather than the query text level, which matters when agents bypass SQL entirely. The testing is also *decentralized*: engines run suites in their own CI instead of submitting to a central lab. And the output is *category-granular* rather than a binary pass/fail verdict, because an agent deciding where to route a plan cares about which operators work, not whether the engine "passed" some aggregate bar.

7 Discussion

The obvious weakness is self-reporting bias. Engines can game their results, and we have no mechanism to prevent it yet. Cryptographic attestation is on the roadmap, though it brings its own complications around key management. Coverage is another concern: 1,635 tests sound like a lot until you consider the space of possible plans. The 12 failing TPC-DS queries are a useful reminder that some failures may not be "bugs" at all. Recursive CTEs, certain `ROLLUP`/`GROUPING SETS` patterns, temporal casts: these are areas where the Substrait specification has not settled on a single correct answer.

Environmental variation (hardware, OS, compiler) can confound cross-engine comparison. We considered mandating Docker-based execution but decided against it: the added complexity would raise the onboarding barrier, and the whole point of decentralized compliance is that engines test in their own environments. Leaderboard rankings could also be misread as performance rankings; we deliberately avoid timing data and use "compliance" rather than "benchmark" throughout, but the risk of conflation persists [5].

The routing evaluation is small (50 items, two engines in the routing pool, threshold-based policy). The 5 residual string failures show that the router cannot help when no engine is adequate in a category. A production router would need plan decomposition across engines, dynamic evidence updates as engines ship new versions, and latency-aware selection that balances correctness against execution cost.

More broadly, decentralized conformance testing raises governance questions our framework surfaces but does not answer. Who decides when a disputed test case is "correct"? How should the community handle selective category reporting? What happens when an agent routes based on stale data from three releases ago? These are socio-technical problems that will matter more as the ecosystem grows. We suspect the answers will look more like standards-body processes than software releases, but we do not yet have enough operational experience to say with confidence.

8 Conclusion

We set out to answer a narrow question: can decentralized, category-level conformance testing produce evidence that agents can actually use for routing decisions? The answer, based on testing three production engines, is a qualified yes. The compliance profiles we observed are not uniform. Engine A and Engine B pass every TPC-H query semantically; Engine C parses them all but stumbles on execution. TPC-DS reaches 87.9%. The 94% aggregate function pass rate hides category swings large enough to change routing decisions. When we fed this evidence to a prototype router, it reduced mismatches from 15 to 5 (66.7%), avoiding all 10 TPC-DS failures by routing away from an engine with only extrapolated evidence.

The 5 residual string failures are equally instructive. Both engines score below 60% on string operations, well under the 85% threshold. The router correctly identified the gap but could not fix it. Compliance-aware routing helps where evidence discriminates between engines; it cannot compensate for categories where the ecosystem itself is immature. That distinction matters for setting expectations about what this kind of infrastructure can and cannot do.

Engine B's trajectory from 2/22 to 22/22 on TPC-H may be the most practically significant result: the framework did not just reveal a gap, it helped close one. The Calcite analogy [1] feels apt. Calcite gives heterogeneous backends a shared optimization layer without requiring them to surrender execution control. Our framework does something similar for validation.

What comes next? Expanding TPC-DS coverage, integrating Engine D [8], scaling the routing evaluation, and feeding conformance evidence into cost-based optimizers [15].

Acknowledgments

We thank the SAO Workshop organizers for the opportunity to discuss these ideas.

References

[1] Edmon Begoli, Jesús Camacho-Rodríguez, Julian Hyde, Michael J. Mior, and Daniel Lemire. 2018. Apache Calcite: A Foundational Framework for Optimized Query Processing Over Heterogeneous Data Sources. In *Proc. SIGMOD 2018*. ACM, 221–230.

[2] Databricks. 2025. Trustworthy AI in the Agentic Lakehouse. arXiv preprint arXiv:2511.16402.

[3] Cynthia Dwork. 2006. Differential privacy. In *Automata, Languages and Programming*, 1–12. Springer.

[4] Andrew Eisenberg and Jim Melton. 1999. SQL:1999, formerly known as SQL3. *ACM SIGMOD Record* 28, 1 (1999), 131–138.

[5] Jim Gray. 1993. *The Benchmark Handbook for Database and Transaction Processing Systems* (2nd ed.). Morgan Kaufmann.

[6] Alon Halevy. 2005. Why your data won't mix. *Queue* 3, 8 (2005), 50–58.

[7] Paras Jain et al. 2023. Analyzing and Comparing Lakehouse Storage Systems. In *Proc. CIDR 2023*.

[8] Andrew Lamb et al. 2024. Apache Arrow DataFusion: A Fast, Embeddable, Modular Analytic Query Engine. In *Companion of the 2024 International Conference on Management of Data (SIGMOD-Companion '24)*. ACM, 5–17.

[9] Limin Ma, Ken Pu, and Ying Zhu. 2024. Evaluating LLMs for Text-to-SQL Generation With Complex SQL Workload. arXiv preprint arXiv:2407.19517.

[10] Jim Melton and Alan R. Simon. 2001. *SQL:1999: Understanding Relational Language Components*. Morgan Kaufmann.

[11] Andy Pavlo et al. 2024. What Goes Around Comes Around... And Around... *SIGMOD Record* 53, 2 (2024), 21–37.

[12] Pedro Pedreira et al. 2022. Velox: Meta's Unified Execution Engine. *Proc. VLDB Endow.* 15, 12 (2022), 3372–3384.

[13] Meikel Poess and Chris Floyd. 2000. New TPC Benchmarks for Decision Support and Web Commerce. *ACM SIGMOD Record* 29, 4 (2000), 64–71.

[14] Shreya Shankar et al. 2022. Operationalizing Machine Learning: An Interview Study. arXiv preprint arXiv:2209.09125.

[15] Michael Stonebraker and Ugur Çetintemel. 2005. "One Size Fits All": An Idea Whose Time Has Come and Gone. In *Proc. ICDE 2005*. IEEE, 2–11.

[16] Substrait Project. 2026. Substrait: Cross-language Serialization for Relational Algebra. https://substrait.io/.

[17] Jacopo Tagliabue et al. 2025. Supporting Our AI Overlords: Redesigning Data Systems to be Agent-First. In *Proc. CIDR 2026*. arXiv preprint arXiv:2509.00997.

[18] Jacopo Tagliabue et al. 2026. Building a Correct-by-Design Lakehouse. arXiv preprint arXiv:2602.02335.

[19] Transaction Processing Performance Council. 2026. TPC Benchmark H Standard Specification. http://www.tpc.org/tpch/.

[20] Michael Armbrust, Ali Ghodsi, Reynold Xin, and Matei Zaharia. 2021. Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics. In *Proc. CIDR 2021*.